

RAG va vektor ma'lumotlar bazalari

Tabiiy tilni qayta ishlash — 14-ma'ruza

A.A. Abdulali

11:30–12:50 · mavzu №14

8-iyul 2026

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L13 da fine-tuning modelni ma'lum vazifa yoki uslubga moslashtiradi. U modelga **tez yangilanadigan yangi faktlarni** arzon va ishonchli qo'sha oladimi?

2-savol

LLM o'z bilimida yo'q savolga duch kelsa, doim “bilmayman” deydimi yoki manbasiz holda ishonchli, lekin xato javob berishi mumkinmi?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L13 da fine-tuning modelni ma'lum vazifa yoki uslubga moslashtiradi. U modelga **tez yangilanadigan yangi faktlarni** arzon va ishonchli qo'sha oladimi?

2-savol

LLM o'z bilimida yo'q savolga duch kelsa, doim “bilmayman” deydimi yoki manbasiz holda ishonchli, lekin xato javob berishi mumkinmi?

1-javob

Asosiy yechim sifatida **yo'q**. Fine-tuning ko'proq xulq, uslub yoki vazifani sozlaydi; yangi faktlar uchun esa qayta o'qitish qimmat va sekin.

2-javob

Bunday holatda **gallyutsinatsiya** xavfi bor. RAG yechimi: avval tashqi manbadan tegishli hujjatni qidiradi, keyin javobni shu kontekstga tayantiradi.

Dars yakunida nimalarni bajara olasiz?

- 1 Katta til modeli (**LLM**, *Large Language Model*) cheklovlarini — **gallyutsinatsiya** va **model bilimining eskirishini** tushuntira olasiz.
- 2 RAG (*Retrieval-Augmented Generation*) nima uchun javobni tashqi manbaga tayantirishini izohlay olasiz.
- 3 RAG prompt token byudjetini hisoblay olasiz:
$$3 \text{ ta bo'lak} \times 200 + 100 = 700 \text{ token.}$$
- 4 Vektor embeddinglar orqali semantik qidiruv qanday ishlashini tushuntira olasiz: **kosinus o'xshashlik** va **top-k** natijalar.
- 5 Vektor ma'lumotlar bazalari RAG tizimida qanday rol bajarishini farqlay olasiz: ular ANN indekslar orqali top-k qidiruvni tezlashtiradi.

Oldindan kerak bo'ladigan bilimlar

L13: fine-tuning L3/L6: embedding va kosinus o'xshashlik L4: o'xshashlik qidiruvi (LSH) Matematika: skalyar ko'paytma, ulush.

Bugungi dars rejasi

Bosqich	Mavzu	Kutiladigan natija
1	LLM cheklovlari va RAG g'oyasi	LLM cheklovi: gallyutsinatsiya + eski bilim
2	RAG jarayoni: qidirish, boyitish, yaratish	prompt byudjeti: 700 token
3	Vektor embeddinglar va semantik qidiruv	kosinus o'xshashlik, top- <i>k</i> natija
4	Vektor ma'lumotlar bazalari	katta bazada tez top- <i>k</i> qidiruv
Xulosa va 14-amaliyotga ko'prik		mini-RAG tizimiga tayyorgarlik

Bu dars RAG tizimiga olib keladi

Tashqi bilimni qidirib, LLM javobini shu manbaga tayantiramiz. Bugungi 700 tokenlik prompt hisobi 14-amaliyotda kod orqali tekshiriladi:

```
assert prompt_tokens == 700
```

Bugungi muammo: LLM manbasiz javob berishi mumkin

Faqat LLM qayerda qiynaladi?

Gallyutsinatsiya

LLM ba'zan yetarli manba bo'lmasa ham **ishonchli ko'rinadigan**, lekin xato javob yaratishi mumkin. Huquq, tibbiyot va rasmiy hujjatlarda bu xavfli.

Model bilimining eskirishi

Modelning ichki bilimi o'qitish davridagi ma'lumotlarga bog'liq. Yangi qonun, yangilik yoki ichki hujjat bo'yicha javob berishda tashqi manba kerak bo'lishi mumkin.

Bugungi yechim: RAG

- 1 **Qidirish** (*retrieve*): bilim bazasidan tegishli hujjat bo'laklarini topish.
- 2 **Boyitish** (*augment*): topilgan matnni promptga kontekst sifatida qo'shish.
- 3 **Yaratish** (*generate*): javobni shu kontekstga tayangan holda yaratish.

RAG g'oyasi

RAG (*Retrieval-Augmented Generation*) — qidiruv bilan boyitilgan javob yaratish yondashuvi:

qidirish → boyitish → yaratish.

Agar **tegishli va ishonchli manba** topilsa, javob shu manbaga tayangan bo'ladi va gallyutsinatsiya xavfi kamayadi.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: yopiq imtihon va ochiq kitobli imtihon

Faqat LLM: yopiq imtihon

Model javobni asosan **ichki bilimiga** tayanib yaratadi. Agar kerakli fakt yetishmasa, u **ishonchli ko'rinadigan**, lekin noto'g'ri matn hosil qilishi mumkin (**gallyutsinatsiya**).

Nega bu muhim?

LLM matnni ehtimollik asosida yaratadi; u faktning rostligini mustaqil tekshiruvchi tizim emas. RAG esa javobni **topilgan manbaga tayantiradi** va tekshirish imkonini beradi.

RAG: ochiq kitobli imtihon

Avval tashqi manbadan kerakli sahifa yoki hujjat bo'lagi **qidiriladi**; keyin LLM javobni shu topilgan kontekstga tayanib yozadi.

L13 bilan bog'lanish

Fine-tuning model xulqi, uslubi yoki vazifasini moslaydi.

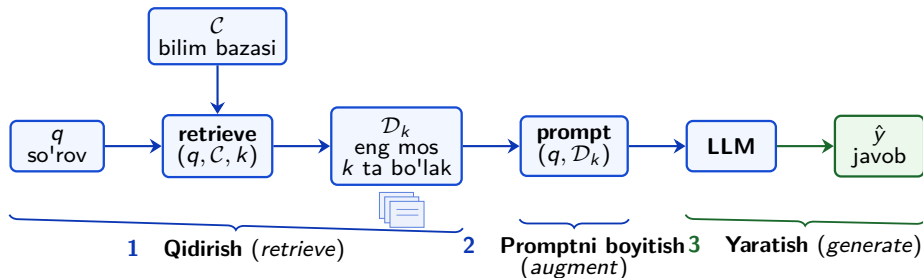
RAG esa modelni qayta o'qitmasdan, tashqi faktni promptga kontekst sifatida keltiradi.

Qisqa qoida: uslub/xulq → fine-tuning; tez yangilanadigan fakt/bilim → RAG.

RAG: rasmiy ta'rif

Ta'rif: RAG (*Retrieval-Augmented Generation*)

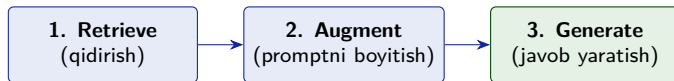
$$\mathcal{D}_k = \text{retrieve}(q, \mathcal{C}, k), \quad \hat{y} = \text{LLM}(\text{prompt}(q, \mathcal{D}_k))$$



Belgilar: q — so'rov; $\mathcal{C}$ — bilim bazasi; $\mathcal{D}_k$ — so'rovga eng mos k ta bo'lak; `prompt( $q, \mathcal{D}_k$ )` — so'rov + topilgan bo'laklar + ko'rsatma; $\hat{y}$ — kontekstga tayangan javob.

Muhim: klassik RAGda model vaznlari o'zgarmaydi; bilim promptga topilgan kontekst orqali keladi. Yangi hujjat odatda bo'laklanadi, embedding olinadi va indeksqa qo'shiladi; modelni qayta o'qitish shart bo'lmasligi mumkin.

RAG uch bosqichi: retrieve → augment → generate



- **Retrieve:** savolga semantik jihatdan eng **mos** hujjat bo'laklarini topish (**top- k** = eng yaxshi k ta natija).
- **Augment:** topilgan kontekst bilan **promptni boyitish** (savol + bo'laklar + ko'rsatma).
- **Generate:** LLM javobni **shu kontekstga tayangan holda** yaratadi.

Muhim g'oya: RAG faqat “qidirish + javob” emas. **Augment** bosqichi — ya'ni topilgan bo'laklarni promptga to'g'ri joylashtirish — alohida va hal qiluvchi qadam.

Nega qidiruv, fine-tuning emas?

Vaziyat

Maqsad: model bugungi qonun, yangi ichki hujjat yoki tez o'zgaradigan faktlar asosida javob bera olishi kerak.

Fine-tuning bilan

- Model **vaznlari** qo'shimcha o'qitish orqali o'zgartiriladi.
- Tez-tez yangilanadigan faktlar uchun bu jarayon **sekin va qimmat**.
- Har yangi hujjat uchun qayta tayyorlash talab qilinishi mumkin.
- Javob qaysi manbaga tayanganini ko'rsatish qiyinroq.

RAG bilan

- Yangi hujjat **bilim bazasiga** qo'shiladi.
- So'rov kelganda mos bo'laklar **qidiriladi**.
- Topilgan kontekst promptga beriladi; model qayta o'qitilmaydi.
- Javobni manba orqali tekshirish osonroq bo'ladi.

Xulosa: xulq, uslub yoki formatni moslashda fine-tuning foydali; **tez yangilanadigan faktlar va domen bilimi** uchun esa RAG odatda qulayroq. Keyingi qadam: prompt byudjetini hisoblash.

Qo'lda misol: bilim kesilishi (*knowledge cutoff*) va RAG

Bilim kesilishi (*knowledge cutoff*)

Faraz qilaylik, LLM 2023-yilgacha bo'lgan ma'lumotlar bilan o'qitilgan. Savol: “2025-yilgi yangi qoidani tushuntiring.”



Faqat LLM

2025-yildagi yangi qoida modelning ichki bilimida bo'lmasligi mumkin. Shuning uchun manbasiz javobda **xato yoki to'qima fakt** xavfi yuqori.

RAG bilan

RAG 2025-yilgi hujjatni bazadan topib, promptga qo'shadi. Javob **manbaga tayan-gan** bo'ladi; tekshirish osonlashadi. Bu kafolat emas: retrieval va manba sifati muhim.

Mini-mashq: qaysi holatda RAG kerak?

Savol

Quyidagi ikki holatdan qaysi biri uchun RAG asosiy yechim bo'ladi?

- (a) model javoblarini rasmiy va muloyim uslubga moslash;
- (b) kompaniyaning bugungi ichki hujjatlari asosida savollarga javob berish.

Mini-mashq: qaysi holatda RAG kerak?

Savol

Quyidagi ikki holatdan qaysi biri uchun RAG asosiy yechim bo'ladi?

- (a) model javoblarini rasmiy va muloyim uslubga moslash;
- (b) kompaniyaning bugungi ichki hujjatlari asosida savollarga javob berish.

Javob

(b) — RAG uchun mosroq holat. Chunki bu yerda javob **tez yangilanadigan tashqi faktlar** va hujjatlarga tayanishi kerak.

Qoida: fakt/bilim/hujjat → RAG; xulq/uslub/format → prompt yoki fine-tuning.

Eslatma: fine-tuning model xulqini barqaror moslaydi; RAG esa yangi yoki o'zgaruvchan bilimni promptga tashqi kontekst sifatida olib keladi.

Kod ↔ formula: RAG skeleti

```
def rag_answer(query, store, llm, k=3):  
    # 1) qidirish (retrieve)  
    docs = store.search(query, k=k)  
  
    # 2) promptni boyitish (augment)  
    context = "\n\n".join(docs)  
    prompt = (  
        "Faqat kontekstga tayanib javob ber.\n"  
        f"Kontekst:\n{context}\n\n"  
        f"Savol: {query}"  
    )  
  
    # 3) javob yaratish (generate)  
    return llm.generate(prompt)
```

Kod → formula:

Kod	Formula
search	$\mathcal{D}_k = \text{retrieve}(q, \mathcal{C}, k)$
prompt	$\text{prompt}(q, \mathcal{D}_k)$
generate	$\hat{y} = \text{LLM}(\text{prompt}(q, \mathcal{D}_k))$

Amaliy bog'lanish

14-amaliy darsida shu uch qadam quriladi: qidirish, promptni boyitish va javob yaratish.

RAG tuzog'i: yomon qidiruv $\Rightarrow$ xato javob xavfi

Asosiy g'oya: RAG sifati = **qidiruv sifati** + **manba sifati**.

Zanjir: yomon *retrieve* $\rightarrow$ noto'g'ri yoki zaif kontekst $\rightarrow$ xato javob xavfi.

Muammo

Agar qidiruv savolga mos bo'lmagan yoki ishonchsiz hujjatlarni qaytarsa, promptga **noto'g'ri kontekst** kiradi. LLM javobni shu kontekstga tayantirishi mumkin; natijada RAG sifati pasayadi.

Nimani nazorat qilamiz?

- **Manba sifati:** ishonchli va yangilangan hujjatlar.
- **Bo'laklash** (*chunking*): mazmun uzilib qolmasin.
- **Embedding va top-k:** eng mos bo'laklar tanlansin.
- **Qayta tartiblash** (*reranking*): nomzodlar saralansin.

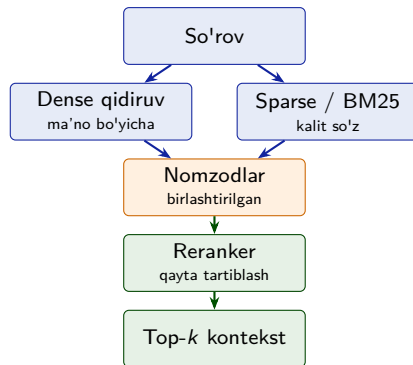
Xulosa: RAG javobni avtomatik ravishda to'g'ri qilmaydi; u **sifatli qidiruv** va **ishonchli manbaga** tayanadi. Shuning uchun endi semantik qidiruv va embeddinglarni ko'ramiz.

RAG sifatini oshirish: hybrid retrieval + reranking

Asosiy g'oya

RAG sifati ko'p jihatdan **qidiruv sifatiga** bog'liq. Amalda ko'pincha bitta retriever emas, balki **gibrid qidiruv** va **qayta tartiblash** birga ishlatiladi.

- **Dense retrieval** — embedding orqali **ma'no bo'yicha** qidiradi.
- **Sparse / BM25** — **aniq moslikni** ushlaydi: kalit so'z, termin, modda raqami.
- **Reranking** — dastlabki nomzodlarni qayta baholab, eng tegishlisini yuqoriga chiqaradi.



Xulosa: real RAGda ko'pincha **nomzod topish** (*dense + sparse*) → **qayta tartiblash** (*reranking*) zanjiri ishlatiladi. Maqsad — promptga **eng tegishli** bo'laklarni kiritish.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish**
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

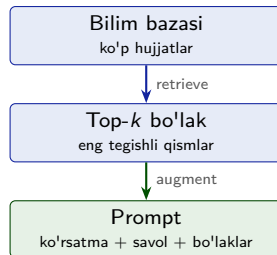
Intuitsiya: kontekst oynasiga faqat keraklisi kiradi

Muammo

LLM kontekst oynasi cheklangan: butun bilim bazasini promptga joylab bo'lmaydi. Ortiqcha token narx, kechikish va shovqinni oshiradi.

RAG yechimi

Avval savolga eng mos bo'laklar qidiriladi. **Top-k** — eng mos k ta bo'lak. Promptga faqat shu bo'laklar qo'shiladi.



$$T_{\text{prompt}} \approx T_{\text{instr}} + kT_{\text{chunk}}$$

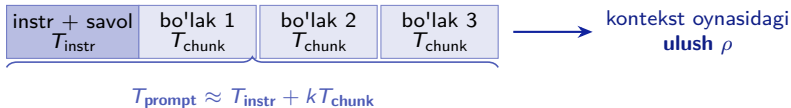
Bu yerda T_{instr} — ko'rsatma, savol va prompt qolipi tokenlari. Prompt uzunligi k va bo'lak hajmiga bog'liq.

RAG prompt byudjeti: rasmiy ta'rif

Ta'rif: prompt token hisobi

$$T_{\text{prompt}} = T_{\text{instr}} + \sum_{i=1}^k T_i \approx T_{\text{instr}} + kT_{\text{chunk}}, \quad \rho = \frac{T_{\text{prompt}}}{T_{\text{window}}}$$

T_{instr} — ko'rsatma + savol + prompt qolipi tokenlari.



Belgilar: k — bo'laklar soni; T_i — i -bo'lak uzunligi; T_{chunk} — bir bo'lakning o'rtacha token soni; T_{window} — kontekst oynasi.

Maqsad: T_{prompt} oynaga sig'sin, narx va kechikish nazoratda bo'lsin.

Keltirib chiqarish: prompt nechta tokendan iborat?

Asosiy g'oya

RAG prompti ikki asosiy qismdan tuziladi:

ko'rsatma + savol + qolip + topilgan k ta bo'lak.

- 1 Retrieve k ta bo'lak qaytaradi: $\mathcal{D}_k = \{d_1, \dots, d_k\}$.
- 2 Agar har bir bo'lak uzunligi T_i token bo'lsa, bo'laklar jami:

$$\sum_{i=1}^k T_i.$$

- 3 Ko'rsatma, savol va prompt qolipi tokenlarini bitta had sifatida olamiz:

$$T_{\text{instr}}.$$

- 4 Umumiy hisob:

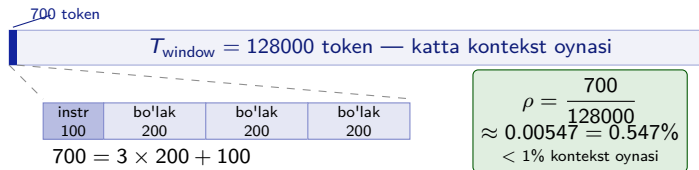
$$T_{\text{prompt}} = T_{\text{instr}} + \sum_{i=1}^k T_i \approx T_{\text{instr}} + kT_{\text{chunk}}.$$

Xulosa: prompt byudjeti k va bo'lak hajmiga chiziqli bog'liq. Endi aniq sonlar bilan hisoblaymiz.

Qo'lda hisob: RAG prompt = 700 token

Berilgan: $k = 3$, $T_{\text{chunk}} = 200$, $T_{\text{instr}} = 100$

$T_{\text{instr}} = 100$ token — ko'rsatma, savol va prompt qolipi. Kontekst oynasi: $T_{\text{window}} = 128000$ token.



Hisob:

$$T_{\text{prompt}} = kT_{\text{chunk}} + T_{\text{instr}} = 3 \times 200 + 100 = \mathbf{700}.$$

$$\rho = \frac{700}{128000} \approx 0.547\% < 1\%.$$

P14 bilan bog'lanish

```
assert prompt_tokens == 700
assert retrieved_docs == 3
```

Xulosa: k yoki bo'lak hajmi oshsa, prompt tokenlari ham chiziqli oshadi.

Mini-mashq: 5 bo'lak uchun prompt byudjeti

Savol

Berilgan:

$$k = 5, \quad T_{\text{chunk}} = 200, \quad T_{\text{instr}} = 100, \quad T_{\text{window}} = 128000.$$

Bu yerda T_{instr} — ko'rsatma, savol va prompt qolipi tokenlari.

Toping:

$$T_{\text{prompt}} \quad \text{va} \quad \rho = \frac{T_{\text{prompt}}}{T_{\text{window}}}.$$

Mini-mashq: 5 bo'lak uchun prompt byudjeti

Savol

Berilgan:

$$k = 5, \quad T_{\text{chunk}} = 200, \quad T_{\text{instr}} = 100, \quad T_{\text{window}} = 128000.$$

Bu yerda T_{instr} — ko'rsatma, savol va prompt qolipi tokenlari.

Toping:

$$T_{\text{prompt}} \quad \text{va} \quad \rho = \frac{T_{\text{prompt}}}{T_{\text{window}}}.$$

Javob

$$T_{\text{prompt}} = kT_{\text{chunk}} + T_{\text{instr}} = 5 \cdot 200 + 100 = \mathbf{1100} \text{ token.}$$

$$\rho = \frac{T_{\text{prompt}}}{T_{\text{window}}} = \frac{1100}{128000} \approx 0.00859 = \mathbf{0.86\%}.$$

Xulosa: ko'proq bo'lak ko'proq qamrov berishi mumkin, lekin token soni, narx, kechikish va shovqin ham oshadi. Shuning uchun k muvozanat bilan tanlanadi.

Kod ↔ formula: prompt byudjeti

```
def prompt_tokens(k, t_chunk, t_instr):  
    return k * t_chunk + t_instr  
  
T = prompt_tokens(k=3, t_chunk=200, t_instr=100)  
window = 128000  
rho = T / window  
  
print(T)          # 700  
print(rho)         # 0.00546875 (~0.547%)
```

Bu soddalashtirilgan hisobda so'rov tokenlari ko'rsatma/qolip tarkibida deb olinadi.

Kod → formula:

Kod	Formula
$k * t_chunk$	$k \cdot T_{\text{chunk}}$
$+ t_instr$	$+ T_{\text{instr}}$
T	T_{prompt}
T / window	$\rho = \frac{T_{\text{prompt}}}{T_{\text{window}}}$

Amaliy dars bilan bog'lanish

Darsda prompt shu byudjet bo'yicha yig'iladi: token soni va retrieved bo'laklar soni tekshiriladi.

RAG tuzog'i: ko'p kontekst har doim foydali emas

"Lost in the middle"

Uzun promptda model ba'zan o'rtada joylashgan muhim ma'lumotdan yomonroq foydalanishi mumkin. Shuning uchun juda ko'p bo'lak qo'shish narx, kechikish va keraksiz shovqinni oshiradi.

Amaliy yechim

- k ni kichik qiymatdan boshlang, masalan 3–5.
- **Manba sifati:** ishonchli va yangilangan hujjat tanlang.
- **Chunking:** matn mazmunli bo'laklarga ajratilsin.
- **Reranking:** nomzodlar qayta tartiblansin.

Ko'p bo'lak $\Rightarrow$ ko'proq token, lekin har doim yaxshi javob emas



Qisqa qoida: ko'proq kontekst emas, **tegishli kontekst** muhim. RAG sifati k soni, bo'lak sifati, manba sifati va tartiblashga bog'liq.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv**
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

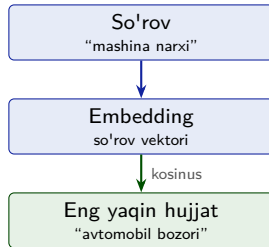
Intuitsiya: so'z emas, ma'no bo'yicha qidiramiz

Kalit so'z qidiruvi

Oddiy qidiruv **“mashina”** va **“avtomobil”** so'zlarini boshqa deb ko'rishi mumkin, chunki u ko'proq so'z mosligiga tayanadi.

Vektor qidiruv

So'rov va hujjatlar **embedding**ga, ya'ni vektor ko'rinishga aylantiriladi. Keyin ma'nosi yaqin bo'lgan vektorlar qaytariladi.

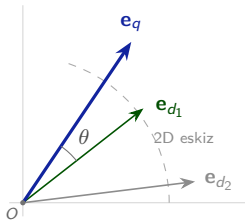


Eslatma: embedding L3/L6 da o'rganilgan. Bu yerda u **qidiruv** uchun ishlatiladi. Kosinus o'xshashlik — vektorlar yaqinligini o'lchash metrikasi.

Semantik qidiruv: rasmiy ta'rif

Ta'rif: kosinus o'xshashlik va top- k qidiruv

$$\text{sim}(q, d) = \cos(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\| \|\mathbf{e}_d\|}, \quad \mathcal{D}_k(q) = \text{TopK}_{d \in \mathcal{C}} \text{sim}(q, d)$$



kichik $\theta \Rightarrow \cos \theta$ katta
 $\Rightarrow$ o'xshashlik yuqori
 $\Rightarrow$ top- k ga kiradi

katta $\theta \Rightarrow \cos \theta$ kichik
 $\Rightarrow$ kamroq o'xshash
 $\Rightarrow$ odatda olinmaydi

Belgilar: $\mathbf{e}_q$ — so'rov embeddingi; $\mathbf{e}_d$ — hujjat embeddingi; $\text{sim}(q, d)$ — kosinus o'xshashlik; $\mathcal{C}$ — hujjatlar to'plami; $\mathcal{D}_k(q)$ — o'xshashligi eng yuqori k ta hujjat.

Intuitsiya: embedding modeli matnni vektorga aylantiradi; qidiruv esa so'rovga **ma'nosi eng yaqin** hujjatlarni topadi.

Davomi: vektor DBda qaysi metrika tanlanadi?

Asosiy fikr

Vektor ma'lumotlar bazasida qidiruv uchun **metrika** tanlanadi: cosine similarity, dot product yoki L2 distance. Tanlov embedding modeli qanday o'qitilganiga bog'liq.

Metrika	Nimani o'lchaydi?	Qachon qulay?
Cosine similarity	Yo'nalish yaqinligi	semantik qidiruvda keng ishlatiladi
Dot product	Skalyar ko'paytma	normallangan embeddinglarda qulay
L2 distance	Evklid masofasi	ayrim ANN indekslarda ishlatiladi

Ehtiyot bo'ling: metrika embedding modeli tavsiyasiga mos tanlanadi. Noto'g'ri metrika top-k reytingini yomonlashtirishi mumkin.

Keyingi bog'lanish: vektor DBlar tanlangan metrika bo'yicha so'rovga eng yaqin top-k vektorlarni tez topadi.

Muhim bog'lanish

Agar vektorlar normallangan bo'lsa:

$$\|\mathbf{e}_q\| = \|\mathbf{e}_d\| = 1$$

unda:

$$\cos(\mathbf{e}_q, \mathbf{e}_d) = \mathbf{e}_q \cdot \mathbf{e}_d.$$

Shuning uchun cosine va dot product reytingi bir xil bo'lishi mumkin.

Normallangan vektorlarda L2 ham cosine bilan bog'liq:

$$\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2\cos(\mathbf{a}, \mathbf{b}).$$

Lekin umumiy holatda bu metrikalar bir xil emas.

Qo'lda misol: kosinus o'xshashlik = $\frac{2}{3}$

So'rov va hujjat vektorlari

$$\mathbf{e}_q = (1, 1, 1, 0), \quad \mathbf{e}_d = (1, 1, 0, 1)$$

lug'at: nlp, qonun, hujjat, sud

Bu soddalashtirilgan o'quv misoli; haqiqiy embeddinglar odatda zich va yuqori o'lchamli bo'ladi.

Hisob:

$$\mathbf{e}_q \cdot \mathbf{e}_d = 1 \cdot 1 + 1 \cdot 1 + 1 \cdot 0 + 0 \cdot 1 = 2,$$

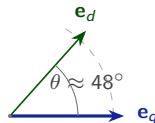
$$\|\mathbf{e}_q\| = \sqrt{1^2 + 1^2 + 1^2 + 0^2} = \sqrt{3},$$

$$\|\mathbf{e}_d\| = \sqrt{1^2 + 1^2 + 0^2 + 1^2} = \sqrt{3},$$

$$\begin{aligned} \text{sim}(q, d) &= \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\| \|\mathbf{e}_d\|} \\ &= \frac{2}{\sqrt{3} \cdot \sqrt{3}} = \frac{2}{3} \approx \mathbf{0.667}. \end{aligned}$$

Talqin

0.667 — musbat va nisbatan yuqori o'xshashlik. Bu hujjat top- k uchun **kuchli nomzod**. Lekin yakuniy o'rin boshqa hujjatlar skori va k qiymatiga bog'liq.



$$\cos \theta = \frac{2}{3} \approx 0.667$$

burchak kichik
 $\Rightarrow$ **yo'nalish yaqin**

Mini-mashq: top- k natijani tanlash

Savol

RAG tizimi hujjat bo'laklari uchun kosinus o'xshashlik skorlarini oldi. Agar $k = 2$ bo'lsa, promptga qaysi ikki bo'lak qo'shiladi?

Bo'lak	Mazmuni	sim
d_1	Yangi qonun bo'yicha asosiy talablar	0.82
d_2	Sud amaliyoti haqida umumiy tarixiy ma'lumot	0.43
d_3	Qonun kuchga kirish sanasi va qo'llanish doirasi	0.76
d_4	Kompaniya marketing rejasi	0.18

Mini-mashq: top- k natijani tanlash

Savol

RAG tizimi hujjat bo'laklari uchun kosinus o'xshashlik skorlarini oldi. Agar $k = 2$ bo'lsa, promptga qaysi ikki bo'lak qo'shiladi?

Bo'lak	Mazmuni	sim
d_1	Yangi qonun bo'yicha asosiy talablar	0.82
d_2	Sud amaliyoti haqida umumiy tarixiy ma'lumot	0.43
d_3	Qonun kuchga kirish sanasi va qo'llanish doirasi	0.76
d_4	Kompaniya marketing rejasi	0.18

Javob

$k = 2$ bo'lgani uchun eng yuqori ikki skor tanlanadi:

$$\mathcal{D}_2 = \{d_1, d_3\}.$$

- d_1 : 0.82 — eng yuqori moslik.
- d_3 : 0.76 — ikkinchi eng yuqori moslik.

Bu ikki bo'lak promptga kontekst sifatida kiradi.

Nega qolganlari kirmaydi?

d_2 va d_4 skori pastroq:

$$d_1(0.82) > d_3(0.76) > d_2(0.43) > d_4(0.18).$$

Past moslikdagi bo'laklar prompt oynasini band qiladi va javobga **shovqin** kiritishi mumkin.

Top- k qoida: bo'laklar moslik skori bo'yicha saralanadi, keyin eng yuqori k tasi promptga qo'shiladi.

Kod $\leftrightarrow$ formula: semantik qidiruv

```
import numpy as np

def cosine(a, b):
    num = a @ b
    den = np.linalg.norm(a) * np.linalg.norm(b)
    return num / den

def top_k(q_vec, doc_vecs, k=3):
    scores = np.array([
        cosine(q_vec, d) for d in doc_vecs
    ])
    order = np.argsort(-scores)
    return order[:k], scores[order[:k]]
```

Kod $\rightarrow$ formula:

Kod	Formula
$a @ b$	$\mathbf{e}_q \cdot \mathbf{e}_d$
$\text{norm}(a)$	$\ \mathbf{e}_q\ $
cosine	$\text{sim}(q, d)$
$\text{argsort}(-\text{scores})$	eng yuqori skorlar
$\text{order}[:k]$	$\mathcal{D}_k(q)$

Amaliy mazmun

Embedding modeli matnni q_vec va doc_vecs vektorlariga aylantiradi. Keyin kod eng yuqori kosinus skorga ega k ta hujjat indeksini qaytaradi.

Masalan, embedding olish uchun sentence-transformers kabi kutubxonalar ishlatilishi mumkin.

Embedding tuzog'i: chunking muhim

Muammo: noto'g'ri bo'lak hajmi

Juda katta bo'lak — bir nechta mavzu aralashib, embedding ma'noni “suyultiradi”.

Juda kichik bo'lak — kerakli kontekst uzilib qoladi. Shuning uchun qidiruv sifati **chunkingga** bog'liq.

Yechim

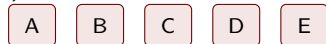
Matnni mazmunan to'liq bo'laklarga ajratamiz: band, paragraf yoki kichik bo'lim. Zarur joyda **overlap** — qo'shni bo'laklar orasida biroz ustma-ust matn qoldiriladi.

1) Juda katta

mavzu A + mavzu B + mavzu C

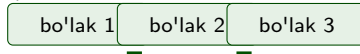
ma'no aralashadi

2) Juda kichik



kontekst uziladi

3) Mos chunk + overlap



chegaradagi ma'no saqlanadi

Qisqa qoida: embeddingdan oldin matnni shunday bo'laklangki, har bir chunk mustaqil o'qilganda ham ma'noli bo'lsin.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari**
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: millionlab vektor ichidan tez topish kerak

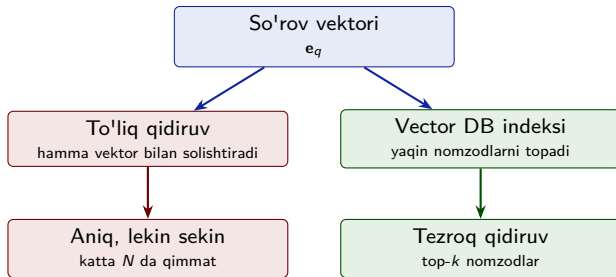
Muammo

Bilim bazasida millionlab bo'lak bo'lsa, har bir so'rovni barcha hujjat vektorlari bilan solishtirish **sekin** bo'ladi. Bu to'liq qidiruvga o'xshaydi:

har bir so'rov $\rightarrow N$ ta taqqoslash.

Yechim

Vektor ma'lumotlar bazasi vektorlarni indekslaydi va so'rovga eng yaqin nomzodlarni tez topadi. Masalan: Pinecone, Weaviate. FAISS esa vektor indekslash/qidiruv kutubxonasi sifatida ishlatiladi.



ANN (*Approximate Nearest Neighbor*) — taxminiy eng yaqin qo'shni qidiruvi. U aniqlik va tezlik o'rtasida amaliy muvozanat beradi.

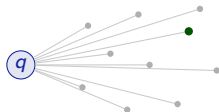
Vektor DB: rasmiy ta'rif

Ta'rif: indeks, upsert, query

$$\text{upsert} : (\text{id}, \mathbf{e}_d, m_d) \longrightarrow \mathcal{I}, \quad \text{query}(\mathbf{e}_q, k) : \mathcal{I} \longrightarrow \mathcal{D}_k(\mathbf{e}_q)$$

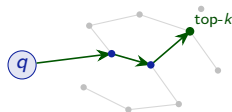
bu yerda $\mathcal{D}_k(\mathbf{e}_q)$ — so'rov vektoriga **taxminan** eng yaqin k ta natija (ANN).

To'liq qidiruv (exact): $O(n)$



tezroq
→

ANN indeks $\mathcal{I}$



har vektor bilan solishtiradi (n ta) faqat nomzodlar bo'ylab yuradi

Belgilar: $\mathcal{I}$ — vektor indeksi (masalan, HNSW); $\mathbf{e}_d$ — hujjat embeddingi; m_d — metadata; $\mathbf{e}_q$ — so'rov embeddingi; $\mathcal{D}_k$ — eng yaqin k ta natija.

Intuitsiya: upsert hujjat vektorlarini indeksga joylaydi, query esa so'rovga mos natijalarni ANN yordamida tez topadi. To'liq qidiruv $O(n)$; ANN esa katta bazalarda amaliyotda ko'pincha ancha tez ishlaydi.

Vektor DB: vositalar va asosiy atamalar

Vositalar: rol bo'yicha

- **Pinecone** — boshqariladigan (*managed*) vektor DB.
- **Weaviate** — open-source vektor DB; cloud varianti ham bor.
- **ChromaDB** — lokal/prototip RAG uchun qulay boshlang'ich vosita.
- **FAISS** — lokal vektor similarity search va indekslash kutubxonasi.

Asosiy atamalar

- **upsert** — vektor, ID va metadataning qo'shish/yangilash.
- **index** — tez top-*k* qidiruv uchun tuzilma.
- **query** — so'rov vektoriga eng yaqin natijalarni topish.
- **metadata filtering** — til, sana yoki teg bo'yicha cheklash.

Muhim: tanlov vazifa, ma'lumot hajmi, byudjet va infratuzilmaga bog'liq. API nomlari farq qilishi mumkin, lekin RAGdagi asosiy mantiq o'xshash:

embedding → upsert/add → query/search → top-*k*.

Nega taxminiy qidiruv (ANN) kerak?

Muammo: aniq qidiruv katta bazada qimmat

Agar bazada N ta vektor bo'lsa, aniq qidiruv har bir so'rovda so'rov vektorini barcha hujjat vektorlari bilan solishtiradi:

aniq qidiruv: N ta taqqoslash $\approx O(N)$.

Millionlab vektorda bu real vaqtli RAG uchun sekin bo'lishi mumkin.

Aniq qidiruv (*exact search*)

- Barcha vektorlar tekshiriladi.
- Natija aniq bo'ladi.
- Katta bazada hisoblash vaqti oshadi.

Taxminiy qidiruv (*ANN search*)

- Indeks orqali istiqbolli nomzodlar ko'riladi.
- Natija taxminiy bo'lishi mumkin.
- Katta bazada qidiruv odatda ancha tezlashadi.

Xulosa: ANN (*Approximate Nearest Neighbor*) barcha vektorlarni tekshirmasdan, indeks orqali yaqin nomzodlarni topadi. Ba'zan ozgina recall/qamrov yo'qolishi mumkin, lekin katta bazalarda tezlik sezilarli oshadi.

Keyingi qadam: sonlar bilan ko'ramiz — aniq qidiruv nechta taqqoslash qiladi, ANN esa taxminan nechta nomzodni tekshiradi?

Qo'lda misol: 10 000 bo'lak, top-3 qidiruv

Bilim bazasi: $N = 10\,000$ vektor

Maqsad: so'rovga eng yaqin **top-3** bo'lakni topish. Quyidagi hisob ANN g'oyasini tushuntirish uchun **soddalashtirilgan nomzodlar soni taqqoslashidir**.

Taqqoslash:

- **To'liq qidiruv** (*brute-force*): har so'rov barcha 10 000 vektor bilan taqqoslanadi:

$$10\,000 \text{ ta taqqoslash} \approx O(N).$$

- **ANN indeks**: barcha vektorlarni emas, taxminiy nomzodlar to'plamini tekshiradi. Masalan, ~ 200 nomzod.

$$\frac{10\,000}{200} \approx 50.$$

$\approx 50 \times$ kamroq nomzod taqqoslash

To'liq qidiruv: 10 000 taqqoslash

ANN: ~ 200 nomzod

Xulosa: brute-force barcha 10 000 vektorni tekshiradi; ANN esa kichik nomzodlar to'plamidan qidiradi. Katta bazalarda bu odatda tezroq, biroq ba'zan recall/qamrov biroz pasayishi mumkin.

Muhim: bu real tezlik kafolati emas. Bu faqat nomzod taqqoslashlar soni bo'yicha soddalashtirilgan nisbat.

Haqiqiy tezlik indeks turi, parametrlar, apparat, embedding sifati va ma'lumot taqsimotiga bog'liq.

Mini-mashq: 1 million vektor uchun ANN nomzodlari

Savol

Berilgan:

$N = 1\,000\,000$ vektor, $M = 1\,000$ ANN nomzod.

To'liq qidiruv barcha N ta vektorni tekshiradi. ANN indeks esa taxminan M ta nomzodni tekshiradi.

Topping: nomzod taqqoslashlar soni to'liq qidiruv bilan solishtirganda necha marta kamayadi?

Mini-mashq: 1 million vektor uchun ANN nomzodlari

Savol

Berilgan:

$N = 1\,000\,000$ vektor, $M = 1\,000$ ANN nomzod.

To'liq qidiruv barcha N ta vektorni tekshiradi. ANN indeks esa taxminan M ta nomzodni tekshiradi.

Topping: nomzod taqqoslashlar soni to'liq qidiruv bilan solishtirganda necha marta kamayadi?

Javob

$$\text{kamayish nisbati} = \frac{N}{M} = \frac{1\,000\,000}{1\,000} = \mathbf{1\,000}.$$

Demak, bu soddalashtirilgan misolda ANN taxminan 1 000 **marta kamroq nomzod** bilan ishlaydi.

Eslatma: bu real tezlik kafolati emas. Haqiqiy tezlik indeks turi, parametrlar, apparat, embedding sifati va ma'lumot taqsimotiga bog'liq. Asosiy g'oya: ANN katta bazada hamma vektorni tekshirmasdan qidiradi.

Kod $\leftrightarrow$ formula: vektor indeksga qo'shish va qidirish

```
# Pseudokod: API nomlari vositaga qarab farq qiladi

# 1) Vektor, ID va metadatani indeksga qo'shish
index.upsert_or_add(
    ids=doc_ids,
    vectors=doc_vecs,
    metadata=doc_meta
)

# 2) So'rov vektoriga eng yaqin top-k natija
hits = index.query_or_search(
    vector=query_vec,
    top_k=3
)

# 3) ID orqali asl matn bo'lagini olish
docs = [store[h.id] for h in hits]
```

Kod $\rightarrow$ formula:

Kod	Formula / ma'nosi
upsert/add	$(id, e_d, m_d) \rightarrow \mathcal{I}$
query/search	$query(e_q, k) \rightarrow \mathcal{D}_k$
hits	top-k ID, skor va metadata
store[h.id]	ID orqali asl matnni olish

Amaliy izoh

Pinecone va Weaviate'da upsert/queryga o'xshash amallar uchraydi. FAISSda esa odatda add/search mantiqi ishlatiladi.

Asosiy mantiq: embeddinglar indeksga ID va metadata bilan qo'shiladi; qidiruvda $\mathcal{D}_k$ topiladi; javob yaratishda esa ID orqali asl hujjat bo'lagi olinadi.

Vektor DB tuzog'i: embedding pipeline mos bo'lishi shart

Muammo: nomuvofiq embedding

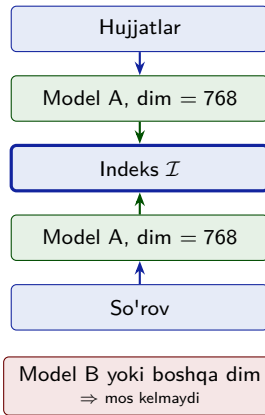
upsert va query bir xil embedding pipeline bilan bajarilishi kerak. Agar model, o'lcham, normalizatsiya yoki preprocessing farq qilsa:

vektor fazosi mos emas $\Rightarrow$ qidiruv sifati pasayadi.

Dimension mos kelmasa, query umuman ishlanmasligi mumkin.

Yechim

Bitta embedding modelini qat'iy saqlang: model nomi, version, dimension, normalizatsiya va preprocessing bir xil bo'lsin. Yangi modelga o'tilsa, indeksni qayta qurish kerak.



Qisqa qoida: indeksga qanday embedding bilan qo'shilgan bo'lsa, query ham aynan shu embedding bilan yuboriladi.

Prezentatsiya rejasi

- 1 Kirish: modelga yangi bilimni qidirib beramiz
- 2 LLM cheklovlari va RAG g'oyasi
- 3 RAG jarayoni: qidirish va yaratish
- 4 Vektor embeddinglar va qidiruv
- 5 Vektor ma'lumotlar bazalari
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: RAG va huquqiy matnlar lex.uz

Nega RAG kerak?

Huquqiy hujjatlar muntazam yangilanadi va javob aniq tahrirga tayanishi kerak. LLM eng yangi tahrirni **bilmasligi** yoki eski ma'lumotga tayanishi mumkin.

Xavf

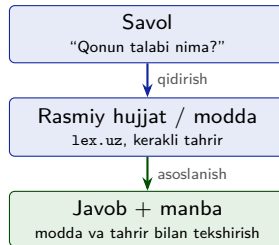
Huquqiy matnda gallyutsinatsiya xavfli: noto'g'ri modda, eski tahrir yoki manbasiz javob noto'g'ri xulosaga olib kelishi mumkin.

O'zbek tili uchun

Qo'shimchalar, so'z yasalishi va imlo variantlari embedding qidiruvga ta'sir qiladi. Shuning uchun model va qidiruv sozlamalari til xususiyatlariga mos bo'lishi kerak.

RAG yechimi

Rasmiy hujjatdan kerakli modda yoki tahrir topiladi, promptga kontekst sifatida qo'shiladi, javob esa **manba bilan** beriladi.



Qisqa qoida: huquqiy RAGda javobning o'zi emas, **qaysi rasmiy manbaga tayangani** ham muhim.

RAGni qanday baholaymiz?

RAG tizimi faqat javobning chiroyli yozilishi bilan emas, **qidiruv sifati** va **javobning manbaga tayanarliligi** bilan baholanadi.

1) Qidiruv sifati (*retrieval*)

- **Recall@k**: kerakli hujjat top- k ichidami?
- **Precision@k**: topilgan bo'laklar savolga mosmi?
- **Manba yangiligi**: hujjat tahriri yoki sanasi to'rimi?

2) Javob sifati

- **Groundedness**: javob topilgan kontekstga tayanadimi?
- **Citation**: manba, modda yoki tahrir ko'rsatilganmi?
- **Hallucination check**: kontekstda yo'q da'vo qo'shilmaganmi?

Xulosa:

RAG bahosi = qidiruv sifati + javobning manbaga sodiqliligi.

Sintez: qachon qaysi yondashuv kerak?

Jihat	Faqat LLM	Fine-tuning (L13)	RAG (L14)
Bilim manbai	ichki bilim	model vaznlari	tashqi kontekst
Nimani o'zgartiradi?	hech narsa	model xulqi/vaznlari	promptga hujjat qo'shadi
Yangi fakt	bilmasligi mumkin	qayta o'qitish kerak	bazaga qo'shib, qidiriladi
Gallyutsinatsiya	yuqori xavf	kamayishi mumkin, lekin kafolat emas	kamayadi, lekin retrieval sifati muhim
Eng mos holat	umumiy savollar	xulq, uslub, format	fakt, hujjat, tez yangilanadigan bilim

Asosiy g'oya

RAG oqimi: embedding bilan qidirish (top-k) → topilgan bo'laklarni promptga qo'shish → javobni kontekstga tayantirib yaratish. Bilim model vaznlariga emas, **prompt kontekstiga** keladi.

Asosiy manba: Lewis va boshqalar (2020)

Manba

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., va boshq. (2020).

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.

NeurIPS 2020.

Asosiy ilmiy hissa

RAG model javob yaratishda ikki turdagi bilimni birlashtiradi:

- **parametrik bilim** — model vaznlarida saqlangan bilim;
- **non-parametrik bilim** — tashqi korpusdan qidirib olinadigan bilim.

Ya'ni generatsiya faqat model ichki xotirasiga emas, **topilgan hujjatlarga** ham tayanadi.

Muhokama savoli: P14 uchun

O'zbek huquqiy matni uchun RAG qursak:

- **Chunking:** modda, band yoki paragraf bo'yicha ajratamizmi?
- **Embedding modeli:** ko'p tilli modelmi yoki o'zbek/yuridik matnga mos modelmi?
- **Til xususiyatlari:** qo'shimchalar, so'z yasalishi va imlo variantlari qidiruvga qanday ta'sir qiladi?
- **Manba:** javobda hujjat nomi, modda va tahrir sanasi qanday ko'rsatiladi?

P14ga ko'prik: amaliyotda hujjat bo'laklanadi, embedding olinadi, indeksga qo'shiladi va savolga mos top- k bo'lak promptga kiritiladi.

Bugungi maqsadlar bajarildi

- 1 ✓ LLM cheklovlari — gallyutsinatsiya, model bilimining eskirishi va RAG nima uchun kerakligini tushuntira olasiz.
- 2 ✓ RAG prompt byudjetini hisoblay olasiz:

$$3 \times 200 + 100 = 700 \text{ token } (< 1\% \text{ kontekst oynasi}).$$

- 3 ✓ Embedding, kosinus o'xshashlik va top- k semantik qidiruv natijalarini talqin qila olasiz.
- 4 ✓ Vektor DBlar va ANN indekslar RAG tizimida qanday rol bajarishini farqlay olasiz.

Asosiy xulosa: RAG — tashqi manbadan tegishli bilimni qidirib, promptga kontekst sifatida qo'shish va javobni shu manbaga tayantirish yondashuvi.

Keyingi ma'ruza (L15): RAG dan agentlarga

Bugungi darsdan ko'prik

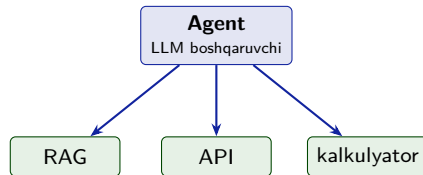
RAG — **bitta kuchli vosita**: bilim bazasidan tegishli hujjatni qidiradi va javobni shu kontekstga tayantiradi. Lekin odatda murakkab vazifani bosqichma-bosqich **re-jalashtirmaydi**.

Murakkab vazifada savol tug'iladi:

- Qachon RAG kerak, qachon API chaqirish kerak?
- Hisob-kitob kerak bo'lsa, kalkulyatorni kim ishlatadi?
- Natijalarni qanday ketma-ketlikda birlashtiramiz?

L15 g'oyasi

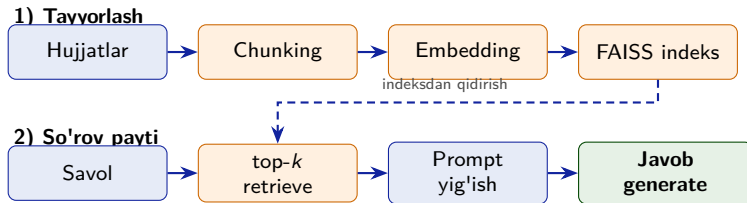
Agent — LLM boshqaruvchi sifatida vositalarni tanlaydi, chaqiradi va natijaga qarab keyingi qadamni belgilaydi. RAG — vositalardan biri; agent — **vositalarni boshqaruvchi**.



Agent vazifaga qarab
vositani tanlaydi

Farq: RAG javob uchun kontekst topadi; agent esa qaysi vositani ishlatishni ham hal qiladi.

Amaliy dars: o'zbek hujjatlari uchun RAG quriladi



To'q-sariq qutilar = RAGEngine (chunking + embedding + FAISS + retrieve); yashil = yakuniy javob.

P14 da qurasiz:

- RAGEngine: chunking + embedding + FAISS + retrieve
- top-k natijadan prompt yig'ish
- javobni **kontekstga tayantirib** yaratish

Birinchi tekshiruvlar:

```
assert prompt_tokens == 700
assert retrieved_docs == 3
```

L14 dagi token hisobi bilan mos:
 $3 \times 200 + 100 = 700$.

- ❶ Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., va boshq. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
- ❷ Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., va Yih, W.-t. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP 2020.
- ❸ Johnson, J., Douze, M., va Jégou, H. (2019). *Billion-scale Similarity Search with GPUs*. IEEE Transactions on Big Data.
- ❹ Malkov, Y. A., va Yashunin, D. A. (2020). *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI).
- ❺ sentence-transformers va FAISS rasmiy hujjatlari.

Ilova S1: chunking strategiyalari

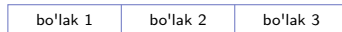
Hujjatni bo'laklarga ajratishning keng tarqalgan usullari:

- **Belgilangan hajm** (*fixed-size*): har N token yoki belgidan kesiladi.
Afzalligi: oddiy. Kamchiligi: gap yoki fikr o'rtasidan bo'linishi mumkin.
- **Strukturaga asoslangan**: gap, paragraf, modda yoki band bo'yicha ajratiladi.
Afzalligi: mazmunan yaxlit bo'lak beradi.
- **Overlap** (*ustma-ust qamrov*): ketma-ket bo'laklar orasida kichik umumiy qism qoldiriladi.
Maqsad: chegaradagi muhim kontekst yo'qolib ketmasin.

Amaliy qoida

O'zbek huquqiy matnlarida ko'pincha **modda/band/-paragraf** bo'yicha bo'lish eng mos keladi; zarur bo'lsa kichik overlap qo'shiladi.

1) Fixed-size



2) Gap/paragraf



3) Overlap



umumiy qism saqlanadi

Ilova S2: HNSW indeksi (qisqacha)

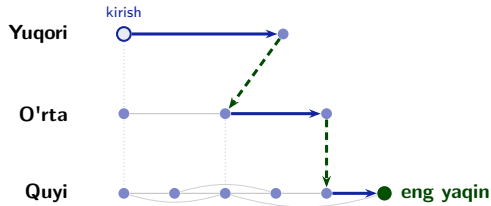
HNSW (*Hierarchical Navigable Small World*) — ANN qidiruvi uchun **ko'p qatlamli graf**.

- **Yuqori qatlamlar** siyrak: uzoq sakrashlar orqali qidiruvni tez yo'naltiradi.
- **Quyi qatlamlar** zich: yaqin qo'shnilar orasida aniqroq tanlashga yordam beradi.
- Qidiruv **yuqoridan** boshlanadi va qatlamma-qatlam **pastga tushadi**.
- Amalda ko'pincha **log-simon tezlashish** beradi, lekin bu qat'iy $O(\log N)$ kafolati emas.

Parametrlar

M — har tugundagi bog'lanishlar soniga ta'sir qiladi: kattaroq M ko'proq xotira, lekin yaxshiroq qamrov berishi mumkin.

ef — qidiruv yoki indeks qurishda ko'riladigan nomzodlar kengligi: kattaroq ef odatda aniqroq, lekin sekinroq.



Yuqori qatlam siyrak, quyi qatlam zich.
Ko'k o'q: qatlam ichida siljish.

Yashil punktir: pastki qatlamga tushish.

Intuitsiya: avval qo'pol yo'nalish topiladi, keyin pastki qatlamlarda yaqin qo'shnilar aniqlashtiriladi.

Ilova S3: zamonaviy RAG kengaytmalari

Bu usullar oddiy RAG pipeline ustiga qo'shilib, qidiruv sifati va kontekst tanlashni yaxshilashga xizmat qiladi.

- ❶ **Query rewriting** — so'rovni qidiruv uchun aniqroq va mosroq shaklga keltirish.
- ❷ **HyDE** (*Hypothetical Document Embeddings*) — LLM avval taxminiy javob yoki hujjat yozadi, keyin shu matn embedding qilinib qidiruvda ishlatiladi.
- ❸ **Self-RAG** — model qachon qidirish kerakligini va topilgan kontekst foydali yoki foydasizligini refleksiya orqali baholaydi.

Ogohlantirish: bular advanced usullar. L14 asosiy oqimida oddiy RAG pipeline yetarli; P14da esa sodda versiya quriladi.